

Desarrollo y bases de datos

1. Arquitectura de la solución

Para dar respuesta a las necesidades de modernización tecnológica de la empresa prototipo descrita en el caso de estudio (un establecimiento de hostelería tradicional con más de 40 años de actividad), se ha diseñado e implementado una arquitectura de tres capas basada en el modelo Cliente-Servidor.

A diferencia de la plataforma global CDS, que emplea un sistema de gestión de contenidos (CMS) genérico para tareas informativas, la solución operativa y transaccional para este negocio (Daily +Dose) se apoya en un desarrollo de software a medida bajo el stack de alto rendimiento LEMP (Linux, Nginx, MySQL, PHP).

1.1. Esquema del Sistema (Arquitectura en Capas)

La infraestructura tecnológica de la solución se estructura en tres niveles modulares independientes, lo que garantiza el aislamiento de funciones y facilita el mantenimiento del sistema:

- **Capa de Presentación (Frontend):** Interfaz web *responsive* desarrollada de forma nativa con HTML5, CSS3 y JavaScript (Vanilla JS). Está diseñada específicamente bajo la filosofía
- *Mobile-First* para garantizar la usabilidad y la ligereza en terminales móviles de sala (empleados) y dispositivos de escritorio (clientes y administración).
- **Capa de Aplicación (Backend):** Lógica de negocio e inteligencia de la aplicación programada en el lenguaje de servidor PHP. El procesamiento de los scripts dinámicos se delega al entorno de ejecución de PHP-FPM (FastCGI Process Manager), optimizando la gestión y el aislamiento de hilos y procesos en el servidor.
- **Capa de Datos (Base de Datos):** Motor relacional MySQL encargado del almacenamiento persistente, estructurado e íntegro de la información del censo de usuarios, flujos de comandas de sala, control de stocks logísticos y el histórico contable del programa de fidelización por puntos.

1.2. Flujo de Información de la Plataforma

El procesamiento de cualquier solicitud de interacción (como la apertura de una comanda, el cambio de estado de una mesa o la consulta y canje del saldo de puntos por parte del cliente) sigue un flujo lineal, asíncrono y seguro:

1. Usuario → Web: El cliente o el empleado interactúa con la interfaz del frontend introduciendo datos de entrada en los formularios interactivos.
2. Web → Servidor (Nginx): El frontend canaliza la petición HTTP hacia el servidor web Nginx a través de conexiones cifradas mediante el protocolo seguro HTTPS.
3. Nginx → PHP-FPM: Al identificar que se solicita un recurso dinámico (un archivo con extensión .php), Nginx no procesa el archivo directamente, sino que redirige de forma eficiente la petición al socket interno de PHP-FPM empleando el protocolo FastCGI.

4. PHP-FPM → MySQL: El entorno backend inicializa la sesión segura, valida los privilegios del rol activo y ejecuta las consultas correspondientes en la base de datos MySQL mediante sentencias preparadas para blindar el sistema.
5. Retorno de datos: El motor de datos devuelve los registros requeridos; PHP procesa la información (o la renderiza) y Nginx sirve el resultado final formateado (HTML/JSON) de vuelta en el navegador del dispositivo cliente sin tiempos de demora.

1.3. Infraestructura de Despliegue y Entorno Real

En concordancia con las pautas de infraestructura cloud y administración de sistemas del proyecto general, el entorno de desarrollo y explotación real se ha configurado utilizando un flujo de trabajo profesional estructurado de la siguiente manera:

- **Entorno de Desarrollo (IDE):** Como estación de trabajo principal en el entorno local (Arch Linux), se ha utilizado Visual Studio Code. Este entorno de desarrollo ligero se ha configurado con extensiones especializadas para el ecosistema PHP, formateadores de código y linters estáticos, agilizando la depuración y la escritura del software dinámico.
- **Control de Versiones (Git):** El código fuente de la aplicación se gestiona, audita e indexa de forma centralizada en un repositorio privado de GitHub, permitiendo la trazabilidad del software y el control estricto de cambios a través de ramas de desarrollo. El repositorio oficial del proyecto está alojado en el siguiente enlace: <https://github.com/Aiizu-27/dailydose.git>
- **Servidor VPS (Virtual Private Server):** El entorno de producción real se aloja en una instancia virtualizada basada en el sistema operativo Linux (Ubuntu Server) dentro de la nube de AWS (Amazon Web Services, concretamente AWS Lightsail). Esta aproximación elimina la necesidad de hardware físico local y garantiza alta disponibilidad básica.
- **Despliegue Seguro (SSH Deploy):** El paso de código al entorno de producción del VPS se realiza mediante el protocolo seguro SSH, utilizando autenticación por clave pública/privada para restringir el acceso remoto exclusivamente a administradores autorizados. El despliegue se ejecuta en caliente en la ruta del servidor web mediante comandos controlados (*git pull*) y una gobernanza estricta de permisos de Linux asignados al usuario del servidor web (*www-data*).

1.4. Justificación Técnica: Desarrollo a Medida frente a Implementación de CMS

La exclusión voluntaria de un Sistema de Gestión de Contenidos (CMS) comercial como WordPress para el núcleo de este caso práctico, optando en su lugar por un desarrollo de código nativo, responde a criterios estrictamente técnicos de administración de sistemas y rendimiento:

- **Eficiencia y Modelado del Motor de Base de Datos:** Los CMS convencionales emplean un modelo de almacenamiento basado en el patrón EAV (Entity-Attribute-Value) a través de tablas maestras unificadas (como *wp_posts* y *wp_postmeta*). Almacenar estructuras complejas como un plano de sala interactivo (**MESAS**), el desglose financiero histórico (**DETALLE_PEDIDO**), o un libro contable virtual (**HISTORIAL_PUNTOS**) en una tabla EAV obligaría al servidor a ejecutar múltiples operaciones de unión (**JOIN**) sobre un mismo repositorio masivo de texto plano, degradando exponencialmente el tiempo de respuesta. El desarrollo desde cero permitió modelar un esquema relacional puro optimizado bajo la Tercera Forma Normal (3FN), garantizando transacciones atómicas y cuadros de caja exactos en milisegundos.

- **Reducción de la Superficie de Ataque (Seguridad Lógica):** Por su masiva adopción global, los CMS comerciales constituyen el principal vector de ataques automatizados en la red. Para dotar a un CMS de funciones de TPV, comandas y sistemas de lealtad, se requiere la instalación masiva de plugins de terceros. Cada extensión añade una nueva dependencia de código ajeno y expande drásticamente la superficie de exposición ante inyecciones o exploits. El desarrollo dinámico nativo elimina estas dependencias; la seguridad se gestiona a nivel interno parametrizando sentencias preparadas contra inyecciones SQL y aplicando middlewares estrictos de sesión para el control de acceso basado en roles (RBAC).
- **Rendimiento de la Infraestructura en Entornos de Alta Concurrencia:** La operativa diaria de un establecimiento hostelero exige inmediatez absoluta. Los CMS cargan en memoria una inmensa infraestructura de archivos y llamadas heredadas para cualquier petición básica, consumiendo un volumen severo de CPU y RAM. La combinación de un frontend nativo en Vanilla JavaScript que consume datos de forma asíncrona mediante la Fetch API, sumado al procesamiento ultraligero de PHP-FPM y la gestión de eventos orientada a hilos de Nginx, reduce drásticamente la huella de memoria del servidor, soportando picos de alta concurrencia en la red local del negocio sin ralentizaciones.
- **Justificación de Competencias Académicas:** El uso de un CMS para resolver el núcleo operativo limitaría la demostración de las competencias adquiridas durante el ciclo formativo. El desarrollo a medida ha permitido proyectar destrezas reales en la configuración avanzada de servidores web (bloques de Nginx, directivas de denegación de acceso y sockets FastCGI), programación de automatizaciones en motores de bases de datos relacionales y el control estricto de permisos en sistemas de archivos Linux.

2. Base de Datos (Caso de Estudio: Daily Dose)

El motor de persistencia diseñado para el caso de estudio de la pyme se asienta sobre un sistema de gestión de bases de datos relacionales (RDBMS) basado en MySQL. Mientras que la infraestructura global del proyecto de la plataforma corporativa CDS delega el almacenamiento secundario en la capa de abstracción intrínseca del CMS WordPress mediante tablas nativas, tipos de contenido personalizados (*Custom Post Types*) y campos avanzados (*ACF*) sin el uso directo de consultas estructuradas, la naturaleza transaccional de la solución de software a medida (Daily Dose) requiere el modelado, estructuración e ingeniería de un esquema relacional puro diseñado íntegramente desde cero.

Este desarrollo nativo e independiente es fundamental para soportar los flujos de comandas en tiempo real, cuadros de caja y el sistema crítico de fidelización por acumulación de puntos sin comprometer la integridad ni el rendimiento del servidor.

2.1. Ingeniería de Datos y Decisiones de Modelado

La arquitectura de datos de la solución se ha diseñado analizando de manera directa las necesidades operativas del establecimiento hostelero tradicional, implementando las siguientes decisiones de ingeniería de software para optimizar la capa de persistencia:

- **Optimización Fina de Tipos de Datos:** Con el fin de reducir el uso de memoria en el servidor virtual (VPS) de AWS Lightsail y acelerar el tiempo de cómputo de los índices, se depuraron los campos numéricos. Se utilizaron tipos eficientes como **SMALLINT** para columnas de volumen controlado como cantidades de artículos y existencias de recompensas, reduciendo el sobrecoste de memoria en disco. Los campos financieros se configuraron de forma estricta bajo el tipo **DECIMAL** para garantizar precisión matemática absoluta y evitar los errores de redondeo de coma flotante comunes en transacciones comerciales.

- **Normalización Estructural del Menú:** Se descartó la práctica no normalizada de almacenar las categorías del menú de la cafetería como meras cadenas de texto embebidas en el catálogo de artículos. En su lugar, se independizó la entidad **CATEGORIAS**, aislando las taxonomías comerciales, permitiendo indexaciones rápidas en las búsquedas del backend y facilitando que el frontend dibuje filtros dinámicos en la interfaz.
- **Aislamiento Contable del Motor de Lealtad:** Las tablas que rigen la acumulación y el canje de la moneda virtual se diseñaron de manera desacoplada de la oferta comercial activa. La entidad transaccional **CANJES** registra de forma imperecedera el coste en puntos aplicado en el instante preciso de la operación, logrando que el histórico contable sea inmune a futuras variaciones de precios, variaciones de stock o modificaciones directas en el catálogo de premios.

2.2. Cumplimiento de las Formas Normales (Normalización)

Para asegurar un diseño relacional óptimo y mitigar anomalías de inserción, actualización o borrado en el tratamiento estructurado de los datos, el esquema se sometió desde su origen a un riguroso proceso de normalización:

- **Primera Forma Normal (1FN):** Se garantizó la atomicidad de todas las columnas. Las propiedades de identidad de los usuarios se segregaron en campos indivisibles (**NOMBRE** y **APELLIDOS**), y se eliminó la posibilidad de almacenar listas de artículos agrupados en un solo registro de comanda mediante el diseño de la tabla de ruptura **DETALLE_PEDIDO**.
- **Segunda Forma Normal (2FN):** Con el esquema en 1FN, se verificó que todos los atributos que no forman parte de la clave primaria dependan funcionalmente de la clave completa. Esto se aplica en las tablas de especialización (**CLIENTES** y **EMPLEADOS**), cuyos atributos específicos (puntos de lealtad o salarios laborales) dependen única y exclusivamente del identificador de entidad correspondiente.
- **Tercera Forma Normal (3FN):** Se eliminaron por completo las dependencias transitivas. En la tabla principal de **PRODUCTOS**, por ejemplo, las filas hacen referencia únicamente al identificador único de la categoría. El nombre descriptivo de dicha categoría reside exclusivamente en su propia entidad, garantizando que un cambio en la etiqueta de una familia de productos no requiera modificar los registros individuales del menú.

2.3. Integridad Referencial y Políticas de Borrado

La consistencia interna y la cohesión relacional de la base de datos se salvaguardan mediante la declaración explícita de restricciones de clave ajena (*Foreign Keys*), asociándolas a comportamientos lógicos automatizados para evitar registros huérfanos ante la eliminación de entidades maestras:

- Políticas **ON DELETE CASCADE**: Implementadas en los nodos críticos de identidad y transaccionales. Si una cuenta raíz es dada de baja en la tabla maestra de **USUARIOS**, sus extensiones de perfil en **CLIENTES** o **EMPLEADOS**, así como sus registros históricos dependientes en **NOTIFICACIONES** o **HISTORIAL_PUNTOS**, se eliminan de manera automática para cumplir con las directivas de privacidad de datos y evitar el almacenamiento de registros basura en el disco del VPS.
- Políticas **ON DELETE SET NULL**: Asignadas a relaciones operativas y de reporting financiero. Si un recurso físico de la sala (como un identificador en **MESAS**) o una ficha técnica variable (como una fila en **ESPECIALIDAD_ACTUAL**) se eliminan del catálogo activo, las cabeceras históricas de los **PEDIDOS** o los campos de los **PRODUCTOS** no se destruyen. La clave ajena se updatea a nulo (**NULL**), manteniendo intacta la contabilidad de las cajas, el histórico de ingresos y la trazabilidad de auditoría del negocio.

2.4. Descripción Detallada de las Entidades del Sistema

El diseño de la base de datos de DailyDose se fundamenta en un modelo relacional normalizado compuesto por 16 entidades distribuidas en cuatro módulos funcionales bien diferenciados:

A) Módulo de Gestión de Acceso y Roles (Bloque Verde)

Este módulo constituye el núcleo de la seguridad lógica y autenticación del sistema, permitiendo la segregación de funciones (RBAC) y la mensajería asíncrona dentro de la aplicación.

- 1. Tabla: USUARIOS
 - Descripción técnica: Repositorio maestro de cuentas de acceso. Governa la autenticación global de la plataforma.
 - Campos clave: **ID_USUARIO** (PK), **NOMBRE**, **APELLIDOS**, **EMAIL**, **CONTRASENA** (almacenamiento en hash criptográfico), **CAMBIAR_PASSWORD** (control de reinicio forzado) y **FECHA_REGISTRO**.
- 2. Tabla: NOTIFICACIONES
 - Descripción técnica: Motor de mensajería del sistema para alertar a los usuarios sobre eventos de forma asíncrona.
 - Campos clave: **ID_NOTIFICACION** (PK), **ID_USUARIO** (FK), **MENSAJE**, **TIPO**, **LEIDO** (booleano) y **FECHA_CREACION**.
- 3. Tabla: CLIENTES
 - Descripción técnica: Extensión del perfil de usuario consumidor, orientada al almacenamiento de datos comerciales y saldos de fidelización.
 - Campos clave: **ID_CLIENTE** (PK), **ID_USUARIO** (FK), **TELEFONO** y **PUNTOS** (moneda virtual acumulada).
- 4. Tabla: EMPLEADOS
 - Descripción técnica: Gestiona la información administrativa y laboral del personal operativo de la cafetería.
 - Campos clave: **ID_EMPLEADO** (PK), **ID_USUARIO** (FK), **PUESTO**, **SALARIO**, **TURNO** y **FECHA_CONTRATACION**.

B) Módulo de Catálogo y Proveedores

Bloque encargado de estructurar el inventario físico, el menú del establecimiento y la trazabilidad logística con terceros.

- 5. Tabla: CATEGORIAS
 - Descripción técnica: Diccionario de clasificación para la organización estructurada de la carta del local.
 - Campos clave: **ID_CATEGORIA** (PK) y **NOMBRE_CATEGORIA**.

- 6. Tabla: PROVEEDORES
 - Descripción técnica: Directorio de entidades externas encargadas del abastecimiento de materias primas.
 - Campos clave: **ID_PROVEEDOR** (PK), **NOMBRE**, **TELEFONO**, **EMAIL** y **DIRECCION**.
- 7. Tabla: PRODUCTOS
 - Descripción técnica: Catálogo comercial maestro de artículos disponibles para su venta al público.
 - Campos clave: **ID_PRODUCTO** (PK), **NOMBRE**, **ID_CATEGORIA** (FK), **ID_ESPECIALIDAD** (FK, mapeado a la ficha técnica del grano), **PRECIO**, **STOCK**, **ID_PROVEEDOR** (FK) y **RUTA_IMAGEN**.
- 8. Tabla: INVENTARIO
 - Descripción técnica: Libro de auditoría de almacén para registrar entradas de stock, mermas e incidencias de mercancía.
 - Campos clave: **ID_MOVIMIENTO** (PK), **ID_PRODUCTO** (FK), **CANTIDAD**, **FECHA** y **TIPO_MOVIMIENTO**.

C) Módulo de Gestión, Operativa y Ventas

Gobernador de la actividad en tiempo real del local, coordinando la ocupación física de la sala, las comandas de barra y su posterior liquidación económica.

- 9. Tabla: PEDIDOS
 - Descripción técnica: Cabecera transaccional que unifica y monitoriza el ciclo de vida de cada venta.
 - Campos clave: **ID_PEDIDO** (PK), **FECHA**, **TOTAL**, **ID_EMPLEADO** (FK), **ID_CLIENTE** (FK), **ID_MESA** (FK) y **ESTADO**.
- 10. Tabla: DETALLE_PEDIDO
 - Descripción técnica: Tabla de ruptura que resuelve la relación N:M entre ventas y productos, garantizando la inmutabilidad de los precios históricos.
 - Campos clave: **ID_DETALLE** (PK), **ID_PEDIDO** (FK), **ID_PRODUCTO** (FK), **CANTIDAD**, **PRECIO_UNITARIO** y **SUBTOTAL**.
- 11. Tabla: PAGOS
 - Descripción técnica: Registro de la liquidación financiera de las comandas para el control de cajas.
 - Campos clave: **ID_PAGO** (PK), **ID_PEDIDO** (FK), **TIPO_PAGO**, **MONTO** y **FECHA**.
- 12. Tabla: MESAS
 - Descripción técnica: Representación en el sistema de la capacidad física de la sala para la gestión operativa de los camareros.
 - Campos clave: **ID_MESA** (PK), **NUMERO_MESA**, **CAPACIDAD**, **UBICACION** y **ESTADO** (**LIBRE**, **OCUPADA**, **RESERVADA**).

- 13. Tabla: ESPECIALIDAD_ACTUAL

- Descripción técnica: Ficha de especialización de atributos técnicos para el café de origen estacional. Evita la presencia de campos nulos en artículos genéricos de la carta.
- Campos clave: ID_ESPECIALIDAD (PK), ORIGEN_GRANO, NOTAS_CATA, TUESTE, METODO_FILTRO, DESCRIPCION_PROCESO, SEASONAL_NAME, SEASONAL_DESC, FECHA_INICIO y FECHA_FIN.

D) Módulo de Fidelización y Recompensas

Controlador del programa de marketing relacional de la marca, asegurando la transparencia total en los balances contables de puntos acumulados y canjeados.

- 14. Tabla: RECOMPENSAS

- Descripción técnica: Catálogo de incentivos y premios disponibles para el intercambio de puntos por lealtad.
- Campos clave: ID_RECOMPENSA (PK), NOMBRE, DESCRIPCION, COSTE_PUNTOS, TIPO, STOCK y SOLO_EMPLEADOS (filtro de seguridad para incentivos internos).

- 15. Tabla: CANJES

- Descripción técnica: Registro histórico inalterable de intercambios de puntos por beneficios físicos.
- Campos clave: ID_CANJE (PK), ID_CLIENTE (FK), ID_RECOMPENSA (FK), FECHA_CANJE y PUNTOS_GASTADOS.

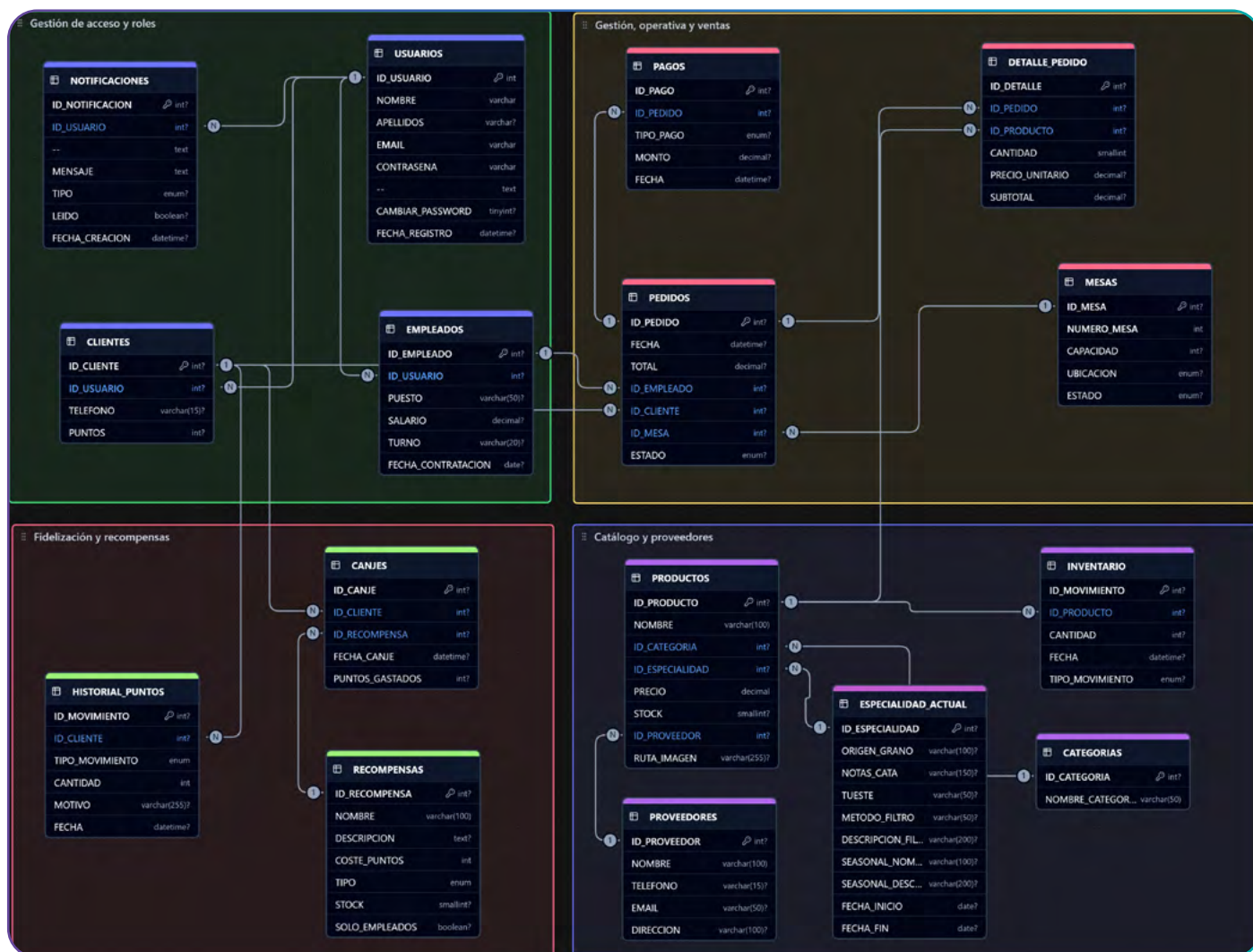
- 16. Tabla: HISTORIAL_PUNTOS

- Descripción técnica: Libro mayor contable encargado de registrar de manera cronológica cada suma o resta de puntos para prevenir fraudes.
- Campos clave: ID_MOVIMIENTO (PK), ID_CLIENTE (FK), TIPO_MOVIMIENTO, CANTIDAD, MOTIVO y FECHA.

2.5. Interconexiones y Relaciones del Modelo Relacional

El mapa relacional diseñado para la capa de datos de la solución de software (ilustrado en la Figura correspondiente a la vista de ChartDB) se articula formalmente a través de las siguientes dependencias funcionales:

- Relaciones Uno a Uno (1:1): Expresadas en la especialización de las cuentas de acceso. Un registro único en USUARIOS genera de manera unívoca una extensión de fila en la tabla CLIENTES o bien en la tabla EMPLEADOS, manteniendo identidades aisladas y limpias de redundancias.
- Relaciones Uno a Varios (1:N): Representan la estructura principal de la lógica operativa. Una fila de la tabla CATEGORIAS engloba N filas en PRODUCTOS; un registro en CLIENTES genera múltiples apuntes cronológicos en el HISTORIAL_PUNTOS y en la tabla de CANJES; y una cabecera en PEDIDOS concentra múltiples líneas independientes dentro del desglosado de DETALLE_PEDIDO.
- Relaciones Varios a Varios (N:M): Resueltas de manera estricta mediante tablas de ruptura normalizadas en 3FN. La relación lógica bidireccional entre la entidad PEDIDOS y la entidad PRODUCTOS queda resuelta físicamente a través de DETALLE_PEDIDO, inyectando claves ajenas compuestas que asocian los productos demandados con su correspondiente orden transaccional.



3. Backend y Lógica de Negocio

El backend de la aplicación diseñada para el caso de estudio de la pyme hostelera se ha desarrollado íntegramente de manera nativa en el lenguaje de servidor PHP. Esta capa actúa como el núcleo operativo e inteligente del sistema, encargándose de procesar las solicitudes del cliente, gobernar el comportamiento de los diferentes perfiles de usuario y coordinar de forma segura el intercambio de datos con el motor de persistencia relacional a través del entorno de ejecución de alto rendimiento de PHP-FPM sobre el servidor web Nginx.

3.1. Conexión Estricta y Abstracción de la Capa de Datos

Para optimizar las transacciones y blindar la infraestructura ante accesos concurrentes en las horas de máxima actividad del local, la comunicación entre PHP y MySQL se realiza mediante la extensión MySQLi, aplicando técnicas avanzadas de aislamiento:

- **Uso de Sentencias Preparadas (Prepared Statements):** Ningún script del backend concatena variables del usuario de forma directa en las cadenas de ejecución. Los parámetros de entrada son vinculados estrictamente mediante marcadores de posición (*bind_param* o *bindParam*), lo que obliga al motor de datos a precompilar la consulta y neutraliza por completo los vectores de ataque por Inyección SQL.
- **Abstracción Arquitectónica de la Lógica:** Con el fin de aliviar la carga de computación del servidor de aplicaciones y acelerar los tiempos de respuesta, el backend de PHP prescinde de sentencias **SELECT**

complejas o múltiples operaciones de unión (**JOIN**) incrustadas en el código de los archivos. El entorno operativo se limita a invocar Procedimientos Almacenados (**CALL sp_...**) para los flujos de escritura transaccional y a consumir Vistas parametrizadas directamente del motor relacional para las lecturas de la interfaz, abstrayendo la inteligencia del negocio directamente en la capa de datos.

3.2. Implementación Lógica de Roles y Permisos (RBAC)

En concordancia con los perfiles funcionales definidos en el diseño funcional de la plataforma global, el backend de la solución a medida discrimina los privilegios de acceso y las capacidades operativas en función del rol asignado de forma unívoca a la identidad autenticada:

- **Administrador (ADMIN):** Dispone de privilegios de superusuario en el entorno. Tiene autorización para ejecutar scripts críticos de control global de la base de datos, gestionar el censo completo de cuentas y auditar los registros de inventario o mermas de stock.
- **Empleado (EMPLEADO):** Perfil operativo asignado al personal técnico y de sala. Habilita el acceso exclusivo a los controladores que gobiernan el estado físico del local (como la apertura o cierre de mesas) y el procesamiento directo de las comandas comerciales de barra y cocina.
- **Cliente (CLIENTE):** Perfil de consumo con acceso estrictamente restringido a su entorno privado de fidelización. El backend bloquea cualquier interacción que no esté vinculada a la consulta de su propio saldo de puntos netos, su libro de movimientos o la solicitud autónoma de recompensas comerciales disponibles.

3.3. Catálogo de Endpoints y Modularidad del Backend

El backend se estructura de forma modular, donde cada componente funcional está aislado en un script PHP especializado que actúa como un punto de entrada o *Endpoint* operativo para resolver las peticiones del frontend:

Recurso / Entidad	Script PHP Asociado	Método HTTP	Operación y Lógica de Negocio Asociada
Login	login.php	POST	Valida las credenciales de acceso, inicializa la sesión global segura y ejecuta las reglas de redirección automática según el rol detectado.
Usuarios	registro.php	POST	Procesa el alta de nuevos clientes en el sistema, sanitiza los campos de identidad y genera su registro inicial con saldo cero en la tabla de fidelización.
Productos	get_producto s.php	GET	Invoca las vistas optimizadas de la base de datos para retornar el catálogo comercial activo segmentado por familias de productos.
Pedidos	crear_pedido. php	POST	Ejecuta de manera atómica el procedimiento almacenado de venta; inserta las cabeceras y desgloses, dispara el trigger de reducción de existencias e incrementa el saldo de puntos del cliente.
Mesas	mesas.php	PUT / POST	Actualiza en tiempo real el estado lógico de ocupación de los espacios físicos del local (LIBRE, OCUPADA, RESERVADA) para la coordinación de la sala.

3.4. Seguridad de la Aplicación y Control de Acceso

Siguiendo las directrices de seguridad y defensa en profundidad especificadas en el proyecto, el entorno backend implementa contramedidas lógicas rigurosas para mitigar riesgos e impedir vulnerabilidades en la aplicación:

- **Hashing de Contraseñas de Nivel Profesional:** Bajo ninguna circunstancia se transmiten o almacenan contraseñas en texto plano dentro del sistema. Durante el registro de usuarios, el backend intercepta la clave y aplica la función nativa `password_hash()` bajo el algoritmo criptográfico Bcrypt, el cual inyecta un factor de coste adaptativo y una sal (*salt*) aleatoria. En el proceso de autenticación, la verificación se delega estrictamente en la función de tiempo constante `password_verify()`, impidiendo ataques de sincronización (*Timing Attacks*) y garantizando que la información almacenada sea irreversible.
- **Control de Acceso mediante Middleware Lógico:** Con el objetivo de restringir el acceso a secciones específicas y garantizar la confidencialidad de la información crítica de la empresa, cada script sensible del backend incorpora una cabecera de control obligatoria. Esta lógica comprueba la existencia de una sesión activa (`session_start()`) y valida de manera estricta que las variables de sesión del usuario coincidan exactamente con el rol autorizado para ese recurso. Si un cliente intenta invocar de forma manual un endpoint administrativo, el middleware detecta la violación de privilegios, destruye la petición y genera una redirección automatizada hacia un entorno controlado de acceso denegado (Código de estado HTTP 403 Forbidden).
- **Aislamiento del Archivo de Conexión:** Para proteger las credenciales maestras de la base de datos contra fallos de exposición de archivos del servidor web, el script que inicializa el objeto de conexión relacional se ha extraído y posicionado físicamente fuera del directorio raíz público (`/var/www/html`) de Nginx. De esta forma, el archivo solo es accesible de manera interna por el intérprete de PHP mediante rutas de inclusión relativas seguras (`require_once`), impidiendo cualquier intento de descarga o visualización directa desde el navegador web de cara al exterior.

4. Interfaz de Usuario (Frontend)

El frontend de la aplicación desarrollada para el caso de estudio (Daily Dose) constituye la capa de presentación con la que interactúan de forma directa los diferentes actores del sistema (clientes, personal de sala y administradores). En perfecta consonancia con los objetivos de digitalización y la metodología de usabilidad descrita en el marco general del proyecto CDS, la interfaz se ha construido mediante un desarrollo web nativo a medida.

Esta decisión de ingeniería de software es crítica para el negocio hostelero: frente a la rigidez y sobrecarga de procesamiento de un CMS convencional, un frontend desacoplado y optimizado garantiza la velocidad de respuesta, la ligereza de carga en dispositivos móviles bajo la infraestructura Nginx y la adaptabilidad necesaria para digitalizar un entorno operativo de alta concurrencia.

4.1. Tecnologías Empleadas y Filosofía de Diseño

El desarrollo de la interfaz de usuario se asienta sobre estándares web puros, prescindiendo de macro-frameworks reactivos con el fin de mantener un consumo de ancho de banda mínimo en la red local segmentada del establecimiento:

- **HTML5 Semántico:** Utilizado para estructurar el contenido de las pantallas de forma jerárquica y accesible, garantizando que los formularios de pedidos y login cumplan con los estándares de validación nativos.
- **CSS3 Avanzado y Diseño Responsivo:** Implementación de hojas de estilo adaptativas bajo una metodología *Mobile-First*. El diseño visual prioriza la adaptabilidad cromática y dimensional de los componentes, asegurando una visualización idónea tanto en las pantallas de los *smartphones* de los clientes que consultan sus puntos como en las *tablets* de barra y sala utilizadas por los empleados para apuntar comandas.
- **JavaScript Nativo:** Motor encargado de dotar de dinamismo a la interfaz en el lado del cliente. Gestiona la captura de eventos del usuario, gestiona las validaciones locales de formularios antes de su envío al servidor y administra la manipulación directa del Árbol de Objetivos del Documento (DOM).

4.2. Arquitectura de Vistas y Estructura de Módulos

La interfaz pública y privada de Daily Dose se organiza de forma modular mediante un catálogo de vistas especializadas, distribuidas estratégicamente según el rol y la fase del flujo de negocio:

- Módulo de autenticación (*login.php* / *registro.php*): Interfaces limpias orientadas a la captación y al control de acceso seguro. Incorporan validaciones en tiempo real para evitar envíos de campos vacíos o formatos de correo inválidos, funcionando como la pasarela de control perimetral de la aplicación.
- Panel Operativo de Sala (*mesas.php*): Pantalla técnica que representa de forma lógica la distribución física de la cafetería. Muestra mediante componentes gráficos interactivos el estado de ocupación de las mesas (**LIBRE**, **OCUPADA**, **RESERVADA**), permitiendo al personal de sala alterar la disponibilidad espacial de forma visual e instantánea.
- Terminal de Punto de Venta (*pedidos.php*): Interfaz de alta productividad optimizada para el personal de barra. Organiza el catálogo de productos de la base de datos por categorías comerciales, facilitando la adición de artículos a una nota de consumo activa mediante pulsaciones directas, agilizando el flujo de trabajo en las horas punta del local.
- Portal de Fidelización del Cliente (*perfil.php* / *recompensas.php*): Espacio privado de cara al consumidor final. Muestra de forma clara el saldo de puntos acumulados, la progresión del usuario y expone el catálogo de premios disponibles. Los controladores del frontend habilitan o inhabilitan los botones de intercambio dinámicamente comparando el coste de la recompensa con el saldo neto del cliente.
- Ficha Técnica de Especialidad (*especialidad.php*): Vista pública orientada al marketing gastronómico del negocio. Extrae y renderiza dinámicamente las notas de cata, el tipo de tueste y el origen del grano de café estacional, aportando el valor diferencial de la marca.
- Paneles de Gestión Central (*inventario.php* / *admin_usuarios.php*): Entornos de administración avanzados dotados de tablas de datos y formularios para la modificación de existencias de almacén, control de mermas y supervisión del censo de usuarios.

4.3. Flujos de Trabajo e Interacción de Usuario

Para garantizar la eficiencia y evitar la fricción en la actividad diaria del establecimiento tradicional reformado, el frontend define dos flujos de experiencia marcadamente diferenciados:

A) Flujo de Operaciones en Sala (Uso del Personal)

El empleado inicia sesión desde un dispositivo móvil de sala. El frontend lo redirige automáticamente al mapa dinámico de *mesas.php*. Al pulsar sobre un cuadrante asignado a una mesa libre, la interfaz realiza una

transición hacia *pedidos.php*, abriendo una comanda limpia. El camarero selecciona los productos del menú demandados; el frontend calcula visualmente los subtotales y, al pulsar "Confirmar", envía el bloque de datos al backend. Una vez procesado el pago, la interfaz regresa al mapa de mesas, actualizando el estado de la sala y congelando el histórico financiero.

B) Flujo de Cliente y Fidelización (Uso del Consumidor)

El cliente escanea el código QR dispuesto en su mesa física, accediendo a la URL pública de la plataforma. Tras autenticarse, el frontend renderiza su *dashboard* privado en *perfil.php*, mostrando su saldo de puntos. Al navegar al catálogo de recompensas, el usuario interactúa con la interfaz solicitando un premio. El frontend intercepta la acción, lanza una ventana de confirmación y, tras la aprobación del servidor, descuenta visualmente los puntos del marcador en pantalla y genera el código de validación para su entrega en barra, todo ello de forma transparente y sin fricciones.

4.4. Dinamismo de la Interfaz y Consumo Asíncrono de Datos

Para maximizar el rendimiento del servidor web Nginx y proporcionar una experiencia fluida propia de una aplicación de escritorio, el frontend combina dos estrategias de renderizado y comunicación:

1. **Renderizado Dinámico en Servidor (SSR):** Al cargar inicialmente estructuras como el censo en *admin_usuarios.php* o las fichas en *recompensas.php*, el procesador de scripts PHP del backend intercepta la llamada, valida los permisos de rol, extrae la información de las vistas relacionales de MySQL y genera el código HTML con los datos incrustados antes de transferirlo por la red. Esto asegura que la información estructural crítica viaje completamente formada y protegida.
2. **Peticiones Asíncronas mediante Fetch API (AJAX):** Para interacciones operativas que requieren inmediatez (como la actualización del estado de una mesa, la adición de un producto a la comanda o la validación instantánea del login), el frontend evita recargar la interfaz completa. JavaScript intercepta el evento del formulario, encapsula los datos en formato JSON y realiza una petición en segundo plano mediante la directiva *fetch()* hacia los endpoints del backend. Al recibir la respuesta del servidor, JavaScript modifica exclusivamente el nodo específico del DOM afectado (por ejemplo, cambiando el color del botón de la mesa de verde a rojo). Esta técnica optimiza drásticamente el tráfico de la red WiFi segmentada del local y dota al sistema de una velocidad de ejecución crítica para el sector hostelería.